MCPShield Study Guide — Model Context Protocol & Project Mastery

> ***Purpose:*** *A comprehensive 15+ page guide to master the Model Context Protocol (MCP) and the MCPShield project for the workshop event.*

Table of Contents

1. What is MCP?
2. MCP Architecture & Protocol
3. Transport Layer: STDIO vs SSE
4. MCP in MCPShield
5. Project Anatomy
6. The 11 MCP Tools
7. Security Engine: 21 Rules
8. Scoring Engine
9. Code Generators
10. Slack Bot Commands
11. API Endpoints
12. Dashboard Walkthrough
13. Development Workflow
14. Adding a New Security Rule
15. Plugin Architecture & Cloud-Agnostic Design

---

1. What is MCP?

Definition

Model Context Protocol (MCP) is an open protocol developed by Anthropic that standardizes how AI applications (LLMs) communicate with external tools and data sources. Think of it as "USB-C for AI" — a universal connector between AI models and the systems they interact with.

Core Concepts

Concept	Description	MCPShield Example
Host	The AI application that initiates connections	Slack bot / Dashboard chat
Client	A connector that maintains a 1:1 connection with a server	`MCP Client` in agent and API
Server	A program that exposes tools, resources, and prompts	`MCP Server` with 11 tools
Tool	An executable function the AI can call	`scan_environment`, `list_findings`
Resource	Data that can be read by the AI (like files)	(Not used in MCPShield)
Prompt	Pre-written templates for the AI	(Not used in MCPShield)

The MCP Handshake

```
1. Client connects to Server (via STDIO or SSE)
2. Client sends `initialize` request
3. Server responds with capabilities (tools, resources, prompts)
4. Client sends `tools/list` to discover available tools
5. Client sends `tools/call` with tool name + arguments
6. Server executes and returns result
```

Why MCP Instead of Direct API Calls?

- **Security Boundary** — The AI never gets raw API credentials
- **Abstraction** — Tool interface stays stable while underlying APIs change
- **Observability** — Every tool call can be logged, rate-limited, and audited
- **Interoperability** — Any MCP-compatible client can use any MCP server

---

2. MCP Architecture & Protocol

JSON-RPC 2.0

MCP uses **JSON-RPC 2.0** as its wire protocol. Every message is a JSON object with:

```json
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "scan_environment",
    "arguments": {}
  }
}
```

Message Types

Direction	Method	Purpose
Client → Server	`initialize`	Start session, negotiate version
Server → Client	`initialized`	Acknowledge session start
Client → Server	`tools/list`	List all available tools
Server → Client	`tools/list` result	Array of tool definitions
Client → Server	`tools/call`	Execute a specific tool
Server → Client	`tools/call` result	Tool execution result
Server → Client	`notifications/tools/list_changed`	Tell client to refresh tool list

Tool Definition Schema

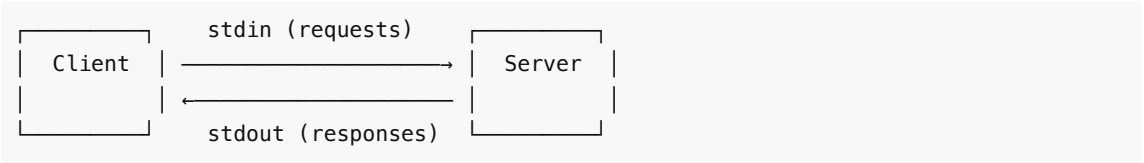
```
{
  "name": "scan_environment",
  "description": "Scan the active AWS environment to catalog resources and detect
misconfigurations.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "services": {
        "type": "array",
        "items": { "type": "string" },
        "description": "Optional subset of AWS services to scan"
      }
    }
  }
}
```

---

3. Transport Layer: STDIO vs SSE

STDIO Transport

The client spawns the server as a child process and communicates via `stdin` / `stdout`.



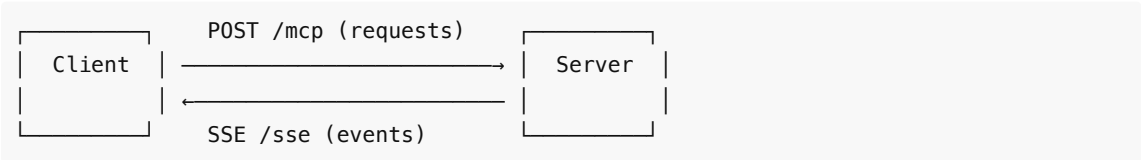
Use case: Local development, same-machine deployment.

MCPShield STDIO Client (agent):

```
transport = new StdioClientTransport({
  command: 'node',
  args: ['../../mcp-server/dist/index.js'],
  env: { ...process.env, MCP_TRANSPORT: 'stdio' },
});
```

SSE Transport (Server-Sent Events)

The client connects to an HTTP endpoint. The server pushes events to the client.



Use case: Docker, remote servers, multi-process deployments.

MCPShield SSE Server:

```
// Fastify SSE endpoint
app.get('/sse', (request, reply) => {
  // Stream events to client
});

app.post('/mcp', (request, reply) => {
  // Handle JSON-RPC requests
});
```

MCPShield Transport Decision

The agent and API check `MCP_TRANSPORT` env var:

Value	Transport	When Used
`stdio`	STDIO	Local `pnpm dev`
`http`	SSE via `MCP_SERVER_URL`	Docker / production

---

4. MCP in MCPShield

Communication Flow

```
User (Slack) ──→ Agent ──→ MCP Client ──→ MCP Server ──→ AWS SDK ──→ AWS/LocalStack
                    ↓                          ↓
                LLM calls                  Security Engine
                                           Scoring Engine
                                           Code Generators
```

Key Security Principle

> ***The AI (LLM) NEVER communicates directly with AWS. ALL operations pass through MCP tools.***

This ensures:

- No raw AWS credentials expose to the LLM
- Every action is logged and auditable
- Human approval gates write operations

Tool Execution Flow

```
1. User: @Shield scan environment
2. Agent receives Slack event
3. Agent calls mcpClient.callTool({ name: 'scan_environment' })
4. MCP Client sends JSON-RPC to MCP Server
5. MCP Server runs scanEnvironment() → runSecurityEngine()
6. MCP Server returns ScanResult
7. Agent formats and sends response
```

---

5. Project Anatomy

Monorepo Structure

```
mcpshield/
├── apps/
│   ├── agent/           # Slack bot + LLM integration
│   ├── api/             # REST API (dashboard backend)
│   ├── dashboard/       # Web dashboard SPA
│   └── mcp-server/      # MCP tool server
├── packages/
│   ├── types/           # Zod schemas + TypeScript types
│   ├── config/          # Env config loader
│   ├── logger/          # Pino logger
│   ├── shared/          # Utilities (IDs, timers, Result)
│   ├── aws-tools/       # AWS SDK clients, scanner, remediator
│   ├── security-engine/ # 21 security rules
│   ├── finding-engine/  # Finding catalog + registry
│   ├── scoring-engine/  # Score calculator
│   ├── llm/             # Shared LLM adapters
│   ├── terraform-generator/  # HCL generation
│   ├── aws-cli-generator/    # CLI generation
│   └── report-generator/     # Report generation
└── docker/              # Dockerfiles
```

Dependency Graph

```
agent  ──→ mcp-client ──→ MCP Server
api    ──→ mcp-client ──→ MCP Server
            ↓
MCP Server ──→ aws-tools ──→ AWS SDK
MCP Server ──→ security-engine
MCP Server ──→ scoring-engine
MCP Server ──→ terraform-generator
MCP Server ──→ aws-cli-generator
MCP Server ──→ report-generator

aws-tools ──→ types
security-engine ──→ types
scoring-engine ──→ types
all ──→ config ──→ types
all ──→ logger
```

Tech Stack

Layer	Technology
Runtime	Node.js 22+, TypeScript strict mode
Package Manager	pnpm workspaces (v11)

MCP	`@modelcontextprotocol/sdk` v1
HTTP	Fastify v5
Validation	Zod
Logging	Pino
Slack	`@slack/bolt` (Socket Mode)
Cloud SDK	AWS SDK v3
Testing	Vitest

---

6. The 11 MCP Tools

Tool Inventory

#	Tool Name	Read/Write	Description
1	`scan_environment`	Read	Scan AWS services for resources and misconfigurations
2	`list_findings`	Read	Retrieve findings from the last scan
3	`describe_finding`	Read	Get full details for a specific finding
4	`security_score`	Read	Compute current security posture score
5	`generate_report`	Read	Generate executive Markdown report
6	`generate_cli_fix`	Read	Generate AWS CLI remediation commands
7	`generate_terraform_fix`	Read	Generate Terraform HCL remediation
8	`approve_remediation`	Write	Approve findings for remediation
9	`execute_remediation`	Write	Apply approved remediations to AWS
10	`rescan_environment`	Read	Re-scan environment (wrapper around scan + score)
11	`health`	Read	Check server connectivity and status

Tool 1: scan_environment

```
Input:  { services?: AwsService[] }
Output: ScanResult {
  scanId: string,
  startedAt: string,
  completedAt: string,
  endpoint: string,
  region: string,
  resourcesScanned: number,
  resourceCounts: Record<string, number>,
```

```
  findings: Finding[],
  score?: SecurityScore
}

Execution:
1. loadState() from disk
2. scanEnvironment(services) → ResourceSnapshot[]
3. runSecurityEngine(snapshots) → Finding[]
4. Merge with existing findings (lifecycle tracking)
5. computeSecurityScore(findings) → SecurityScore
6. saveState(state)
7. Return ScanResult
```

Tool 2: list_findings

```
Input:  { severity?: Severity, service?: AwsService }
Output: { findings: Finding[], total: number }
```

Tool 3: describe_finding

```
Input:  { findingId: string }
Output: { finding: Finding, catalog: FindingCatalogEntry }
```

Tool 4: security_score

```
Input:  {}
Output: SecurityScore {
  score: number,        // 0–100
  grade: 'A' | 'B' | 'C' | 'D' | 'F',
  totalFindings: number,
  breakdown: {
    critical: number,
    high: number,
    medium: number,
    low: number
  },
  computedAt: string,
  delta?: number         // Change from previous score
}
```

Tool 5: generate_report

```
Input:  {}
Output: Report {
  markdown: string,
  score: SecurityScore,
  findings: Finding[],
```

```
  recommendations: Recommendation[]
}
```

Tools 6–7: generate_cli_fix / generate_terraform_fix

```
Input:  { findingId: string }
Output: {
  findingId: string,
  content: string,      // Generated code
  summary: string,
  provider: 'terraform' | 'aws-cli'
}
```

Tool 8: approve_remediation

```
Input:  {
  findingIds: string[],
  approvedBy: string,
  note?: string
}
Output: Approval {
  approvalId: string,
  findingIds: string[],
  status: 'approved',
  approvedBy: string,
  createdAt: string
}
```

Tool 9: execute_remediation

```
Input:  { approvalId: string }
Output: {
  approvalId: string,
  results: RemediationResult[],
  score: SecurityScore
}

Execution:
1. Verify approval exists and is 'approved'
2. For each finding:
   a. Map finding to RemediationAction
   b. Execute via AWS SDK
   c. Mark finding as 'resolved' or 'open'
3. Recompute score
4. Save state
```

Tool 10: rescan_environment

```
Input:  {}
Output: ScanResult  (same as scan_environment)
```

Tool 11: health

```
Input:  {}
Output: {
  status: 'ok' | 'degraded',
  version: string,
  uptimeSeconds: number,
  cloudProvider: {
    reachable: boolean,
    endpoint: string,
    region: string,
    services: Record<string, string>
  },
  lastScanId?: string
}
```

---

7. Security Engine: 21 Rules

Rule Categories

Severity	Count	Rules
🔴 Critical	3	MCPS-S3-001, MCPS-IAM-001, MCPS-IAM-002
🟠 High	7	MCPS-EC2-001, MCPS-EC2-002, MCPS-S3-002, MCPS-S3-003, MCPS-CT-001, MCPS-SSM-001
🟡 Medium	8	MCPS-IAM-003, MCPS-IAM-004, MCPS-IAM-005, MCPS-S3-004, MCPS-LM-001, MCPS-SQS-001, MCPS-SNS-001, MCPS-DDB-001, MCPS-SEC-001
🔵 Low	3	MCPS-TAG-001, MCPS-NAM-001, MCPS-DESC-001

Rule Implementation Pattern

Each rule is a pure function:

```
function checkPublicS3Bucket(snapshot: ResourceSnapshot): RuleEvaluation | null {
  if (snapshot.service !== 's3' || snapshot.type !== 'bucket') return null;

  const pab = snapshot.attributes.publicAccessBlock;
  let isViolated = false;
  const evidence: Record<string, unknown> = {};

  // Check Block Public Access
  if (!pab || !isFullyBlocked(pab)) {
    isViolated = true;
```

```
    evidence.publicAccessBlockMissing = true;
  }

  // Check ACLs for public grants
  // Check policies for wildcard principals

  return { catalogId: 'MCPS-S3-001', isViolated, evidence };
}
```

Rule Functions

#	Function	What It Checks
1	`checkPublicS3Bucket`	S3 Block Public Access, public ACLs, wildcard policies
2	`checkAdminAccessAttached`	AdministratorAccess policy on IAM user
3	`checkOldAccessKeys`	Active keys older than 90 days
4	`checkSshOpenToInternet`	Security group allows 0.0.0.0/0 on port 22
5	`checkRdpOpenToInternet`	Security group allows 0.0.0.0/0 on port 3389
6	`checkS3MissingEncryption`	Bucket has no default SSE
7	`checkS3VersioningDisabled`	Bucket versioning not enabled
8	`checkCloudTrailDisabled`	Trail not logging or not multi-region
9	`checkWeakPasswordPolicy`	IAM password policy below 14 chars / missing complexity
10	`checkUnusedUser`	User inactive > 90 days
11	`checkUnusedAccessKeys`	Keys unused > 90 days
12	`checkS3LoggingDisabled`	Server access logging not configured
13	`checkMissingTags`	Resource missing Owner/Environment/DataClassification
14	`checkNamingConvention`	Bucket name doesn't match pattern
15	`checkMissingDescriptions`	SG/param has no description
16	`checkSsmUnencryptedSecret`	SSM param with sensitive name not SecureString
17	`checkLambdaDeprecatedRuntime`	Lambda on outdated runtime
18	`checkSqsEncryptionDisabled`	SQS queue has no KMS key
19	`checkSnsEncryptionDisabled`	SNS topic has no KMS key
20	`checkDynamoDbDefaultEncryption`	DynamoDB table uses default AWS-owned key
21	`checkSecretsManagerDefaultEncryption`	Secret uses default AWS-managed key

Evidence System

Each rule returns `evidence: Record<string, unknown>` with details about the violation. Example:

```
{
  "staleActiveKeys": [
    {
      "accessKeyId": "AKIA1234567890EXAMPLE",
      "ageDays": 120,
      "created": "2025-03-15T10:00:00Z"
    }
  ],
  "publicSshRules": [{
    "IpProtocol": "tcp",
    "FromPort": 22,
    "ToPort": 22,
    "IpRanges": [{ "CidrIp": "0.0.0.0/0" }]
  }]
}
```

Finding Lifecycle

```
New scan → findings detected
                ↓
          ┌─ existing? ──┐
          │              │
         Yes             No
          │              │
    was 'resolved'?     Add as 'open'
          │
    Yes /    \ No
     'open'  'open'
                ↓
Missing from new scan?
          │
    Was 'open'? → 'resolved'
    Was 'resolved'? → keep 'resolved'
```

---

8. Scoring Engine

Score Calculation

```
Starting score: 100
  - Critical:  -20 each
  - High:      -10 each
  - Medium:     -4 each
  - Low:        -1 each
  Clamp to: 0–100
```

Grade Mapping

Score Range	Grade
90–100	A
70–89	B
50–69	C
25–49	D
0–24	F

Delta Tracking

The scoring engine tracks changes from the previous scan:

```
Previous score: 85 (B)
Current score:  92 (A)
Delta:          +7
```

---

9. Code Generators

Terraform Generator

- Input: `Finding`
- Process: Looks up finding catalog → renders HCL template with resource values
- Template placeholders: `{{bucket}}`, `{{userName}}`, `{{accessKeyId}}`, `{{sgId}}`, `{{region}}`, `{{endpoint}}`

Example output for MCPS-S3-001:

```
resource "aws_s3_bucket_public_access_block" "my-bucket" {
  bucket                  = "my-bucket"
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

AWS CLI Generator

- Same pattern as Terraform but outputs CLI commands
- Uses `--endpoint-url {{endpoint}}` for LocalStack compatibility

Example output for MCPS-S3-001:

```
aws --endpoint-url http://localhost:4566 s3api put-public-access-block \
  --bucket my-bucket \
  --public-access-block-configuration
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBucket
```

Report Generator

- Takes `ScanResult` + `SecurityScore` + all `Finding`s
- Outputs structured Markdown with:
  - Executive summary
  - Score and grade
  - Findings ranked by severity
  - Recommendations table
  - Remediation priority

---

10. Slack Bot Commands

Command Reference

Command	Description	MCP Tool Used
`@Shield scan` or `@Shield scan environment`	Trigger full environment scan	`scan_environment`
`@Shield show findings` or `@Shield findings`	List all findings	`list_findings`
`@Shield show critical` or `@Shield critical`	List critical findings only	`list_findings(severity:critical)`
`@Shield explain finding <id>`	Get LLM-powered explanation	`describe_finding` + LLM
`@Shield explain finding <id> eli5`	Beginner-friendly explanation	`describe_finding` + `eli5.ts`
`@Shield quiz me on finding <id>`	Generate quiz questions	`describe_finding` + `quiz.ts`
`@Shield generate terraform <id>`	Generate Terraform fix	`generate_terraform_fix`
`@Shield generate aws-cli <id>`	Generate AWS CLI fix	`generate_cli_fix`
`@Shield fix finding <id>`	Approve + prepare remediation	`approve_remediation`
`@Shield fix all critical`	Batch approve critical findings	`approve_remediation` (batch)
`@Shield approve`	Execute approved remediation	`execute_remediation`
`@Shield rescan`	Re-scan environment	`rescan_environment`
`@Shield security score` or `@Shield score`	Show current score	`security_score`
`@Shield generate report` or `@Shield report`	Generate executive report	`generate_report`

Other text	Falls through to AI agent	LLM decides which tool to call

Agent Loop

When the LLM requests a tool call:

```
1. LLM returns toolCalls in response
2. Agent executes each tool via MCP client
3. Tool results are fed back to LLM
4. LLM formulates final response
```

---

11. API Endpoints

REST API

Method	Endpoint	Description
GET	`/health`	Server status
GET	`/api/state`	Full state (score, findings, remediations)
POST	`/api/scan`	Trigger scan (delegates to MCP server)
POST	`/api/chat`	AI chat with tool execution

State Endpoint Response

```
{
  "score": {
    "score": 85,
    "grade": "B",
    "totalFindings": 3,
    "breakdown": { "critical": 0, "high": 1, "medium": 1, "low": 1 }
  },
  "lastScan": {
    "scanId": "scn_abc123",
    "resourcesScanned": 45,
    "completedAt": "2026-07-20T10:00:00Z"
  },
  "findings": [ ... ],
  "approvals": [ ... ],
  "remediations": [ ... ]
}
```

Chat Endpoint

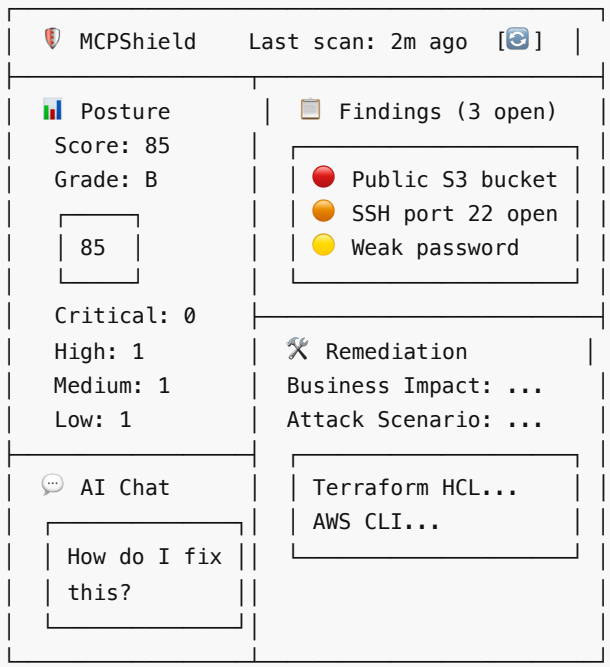
```
POST /api/chat
Body: { message: string, history?: array }
Response: { content: string }
```

```
Flow:
1. Create MCP Client → connect to MCP Server
2. Get LLM provider
3. List MCP tools
4. Call LLM with message + tool definitions
5. If LLM requests tool calls → execute via MCP Client
6. Feed results back to LLM
7. Return final response
```

---

12. Dashboard Walkthrough

Layout



Sections

1. **Header** — Logo, last scan time, refresh button
2. **Posture Card** — Score ring, grade badge, severity counters, resource stats
3. **Findings Card** — Filterable list (all/critical/high/medium/low)
4. **Remediation Card** — Finding detail, tabs (Description/Terraform/CLI), copy buttons
5. **Chat Card** — AI assistant conversation, text input

State Management

The dashboard polls `/api/state` every 5 seconds and re-renders.

---

13. Development Workflow

Local Development

```
# Terminal 1 — MCP Server
pnpm --filter @mcpshield/mcp-server dev

# Terminal 2 — API + Dashboard
pnpm --filter @mcpshield/api dev

# Terminal 3 (optional) — Slack Agent
pnpm --filter @mcpshield/agent dev
```

Build

```
pnpm build          # Build all packages + apps
```

Quality Checks

```
pnpm typecheck      # TypeScript strict mode check
pnpm lint           # ESLint
pnpm format:check   # Prettier
pnpm test           # Vitest
pnpm check          # All of the above
```

Package Management

```
pnpm add <pkg> --filter <workspace>    # Add dependency
pnpm -r build                           # Build all
pnpm --filter @mcpshield/api dev       # Run one app
```

Project Scripts

```
pnpm build    # tsc for all packages
pnpm clean    # Remove dist/ folders
pnpm test     # vitest for all packages
pnpm check    # typecheck + lint + format + test
```

---

14. Adding a New Security Rule

Step-by-Step

1. Add the rule function → `packages/security-engine/src/rules.ts`

```
export function checkNewRule(snapshot: ResourceSnapshot): RuleEvaluation | null {
  if (snapshot.service !== 's3' || snapshot.type !== 'bucket') return null;
  // ... check logic ...
  return { catalogId: 'MCPS-NEW-001', isViolated, evidence };
}
```

2. Register the rule → Add to the `RULES` array

```
export const RULES = [
  checkPublicS3Bucket,
  // ... existing rules ...
  checkNewRule,
];
```

3. Add catalog entry → `packages/finding-engine/src/catalog.ts`

```
{
  id: 'MCPS-NEW-001',
  title: 'New Security Rule',
  severity: 'medium',
  service: 's3',
  category: 'Data Protection',
  description: '...',
  businessImpact: '...',
  technicalImpact: '...',
  attackScenario: '...',
  bestPractice: '...',
  mitre: [{ tactic: '...', techniqueId: 'T...', techniqueName: '...' }],
  cis: [{ benchmark: '...', controlId: '...', title: '...' }],
  baseRiskScore: 50,
  remediation: {
    terraform: '...',
    awsCli: '...',
  },
  references: ['...'],
}
```

4. Add remediation mapping → `apps/mcp-server/src/server.ts`

```
case 'MCPS-NEW-001':
  return {
    findingId: finding.findingId,
    catalogId,
    description: `...`,
    service: 's3',
    operation: '...',
    params: { ... },
  };
```

5. Add dashboard catalog entry → `apps/dashboard/src/app.js`

6. Write tests → `packages/security-engine/src/engine.test.ts`

Rule Registration Options

- Any package exporting a `RuleFunction` type
- Registered in the `RULES` array in `rules.ts`

- Automatically picked up by `runSecurityEngine()`

---

15. Plugin Architecture & Cloud-Agnostic Design

Current State

MCPShield ships with an **AWS provider** only. The architecture is designed for extension:

```
interface CloudProvider {
  name: string;
  scan(): Promise<ResourceSnapshot[]>;
  remediate(action: RemediationAction): Promise<RemediationResult>;
}
```

Adding a New Provider

To add Azure support, you would:

1. Create `packages/azure-tools/` with Azure SDK scanner + remediator
2. Implement `CloudProvider` interface
3. Register in config: `CLOUD_PROVIDER=azure`
4. Add Azure-specific security rules
5. Update code generators for `az` CLI

Provider Configuration

```
CLOUD_PROVIDER=aws        # or azure, gcp
LOCALSTACK_ENDPOINT=...   # Used by AWS provider
AZURE_ENDPOINT=...        # Used by Azure provider
```

Universal Security Rules

Some rules are provider-agnostic:

- Tagging compliance
- Naming conventions
- Encryption at rest
- Audit logging enablement
- Public access controls

These could be shared across providers with provider-specific check implementations.

Appendix A: Key Terms

Term	Definition
MCP	Model Context Protocol — open protocol for AI-tool communication
CSPM	Cloud Security Posture Management — continuous compliance monitoring
JSON-RPC	Lightweight remote procedure call protocol using JSON

SSE	Server-Sent Events — HTTP push technology
STDIO	Standard input/output — process communication channel
HCL	HashiCorp Configuration Language — Terraform syntax
MITRE ATT&CK	Knowledge base of adversary tactics and techniques
CIS Benchmark	Center for Internet Security configuration guidelines
Zod	TypeScript-first schema validation library
Fastify	Fast Node.js HTTP framework
Pino	Low-overhead Node.js logger
Vitest	Unit test framework for Vite projects
pnpm	Fast, disk-efficient package manager

Appendix B: Environment Variables

Variable	Default	Purpose
`NODE_ENV`	`development`	Runtime environment
`LOG_LEVEL`	`info`	Logging verbosity
`LOCALSTACK_ENDPOINT`	`http://localhost:4566`	AWS endpoint
`AWS_ACCESS_KEY_ID`	`test`	AWS credentials
`AWS_SECRET_ACCESS_KEY`	`test`	AWS credentials
`AWS_DEFAULT_REGION`	`us-east-1`	AWS region
`MCP_TRANSPORT`	`http`	Stdio or HTTP SSE
`MCP_HTTP_HOST`	`0.0.0.0`	MCP server bind
`MCP_HTTP_PORT`	`7801`	MCP server port
`MCP_SERVER_URL`	`http://localhost:7801/mcp`	SSE endpoint
`MCPSHIELD_STATE_DIR`	`./.mcpshield-state`	State file directory
`API_HOST`	`0.0.0.0`	API bind address
`API_PORT`	`7802`	API server port
`LLM_PROVIDER`	`nim`	LLM provider selection
`LLM_TEMPERATURE`	`0.1`	LLM temperature
`LLM_MAX_TOKENS`	`1024`	Max response tokens
`NIM_API_KEY`	–	NVIDIA NIM API key

`GEMINI_API_KEY`	–	Google Gemini API key
`SLACK_BOT_TOKEN`	–	Slack bot token
`SLACK_APP_TOKEN`	–	Slack app token

Appendix C: MCPShield File Reference

File	Purpose	Key Functions
`apps/mcp-server/src/server.ts`	MCP tool implementations	`performScanAndSave()`, `mapFindingToRemediation()`
`apps/mcp-server/src/state.ts`	State persistence	`loadState()`, `saveState()`
`apps/mcp-server/src/index.ts`	HTTP/SSE transport	Fastify server, SSE endpoint
`apps/agent/src/slack.ts`	Slack bot handlers	15 command handlers
`apps/agent/src/llm.ts`	LLM provider re-export	`getLlmProvider()`
`apps/agent/src/mcp-client.ts`	Agent MCP client	STDIO + SSE transport
`apps/api/src/index.ts`	REST API endpoints	`/api/state`, `/api/scan`, `/api/chat`
`apps/api/src/mcp-client.ts`	API MCP client	STDIO + SSE transport
`packages/aws-tools/src/scanner.ts`	AWS resource scanning	`scanEnvironment()`
`packages/aws-tools/src/remediator.ts`	AWS remediation execution	`executeRemediationAction()`
`packages/security-engine/src/rules.ts`	21 security rules	All `check*()` functions
`packages/security-engine/src/engine.ts`	Rule orchestrator	`runSecurityEngine()`
`packages/scoring-engine/src/scoring.ts`	Score calculation	`computeSecurityScore()`
`packages/finding-engine/src/catalog.ts`	Finding definitions	21 catalog entries
`packages/llm/src/index.ts`	Shared LLM adapters	`OpenAICompatibleProvider`, `OllamaProvider`
`packages/config/src/index.ts`	Config loader	`loadConfig()`, `getConfig()`